How the SimpleSFT Memory Estimator Works

April 6, 2026

1 What This Report Is Trying To Do

This report explains the memory estimator that is currently implemented in SimpleSFT. It is not a historical survey and it is not a proposal for a future model. The goal is practical:

> Given a model and a training configuration, estimate the peak GPU memory used by one rank during one training step.

The estimator is easiest to understand if you stop thinking about "VRAM" as one giant blob. The code does not predict one opaque number. It builds memory from a small set of named objects, then asks which training phase is worst.

2 The One-Step Mental Model

For memory purposes, one training step has three moments that matter:

1. **Forward:** parameters are resident, some activations are kept for backward, and kernels use temporary workspace.
2. **Backward:** gradients now exist, some forward state is still alive, and backward kernels need their own workspace.
3. **Optimizer step:** gradients and optimizer state remain live while the optimizer fetches parameters, updates them, and communicates with other ranks if training is distributed.

The estimator assigns one candidate peak to each of those moments and then takes the maximum:

$$M_{\text{peak}} = \max(M_{\text{forward}}, M_{\text{backward}}, M_{\text{optimizer}}) .$$

That is the whole top-level idea. Everything else in this report explains where those three candidate peaks come from.

3 The Four Memory Families

Almost every estimator choice fits into one of four buckets:

- **Resident state:** memory that simply exists on the rank, such as parameters, gradients, optimizer state, master weights, and fixed backend support buffers.

- **Retained activations:** forward-pass tensors that are intentionally kept alive because backward still needs them.
- **Workspace:** temporary memory that spikes while a phase is actively running.
- **Carryover:** state that is not part of the clean resident floor but is still hanging around when the optimizer step begins.

This decomposition matters because different training regimes move memory between these buckets. Checkpointing, for example, reduces retained activation state but usually increases backward-time recompute workspace. ZeRO changes the resident state and the optimizer-step windows without changing the model's architecture. LoRA changes which parameters and linears are actually trainable.

4 Why The Estimator Works Per Rank

The estimator always answers a per-rank question. That is the right unit for real training failures because OOM happens on a device, not on the cluster in the abstract.

Two consequences follow immediately:

- Tensor parallelism changes the local width of many tensors.
- Data parallel sharding changes how much of a resident or optimizer object one rank owns.

So two jobs with the same model and sequence length can have very different per-rank peaks if they use different world sizes, tensor-parallel degrees, or ZeRO stages.

5 What The Estimator Takes As Input

The estimator combines two inputs:

- an `EstimatorConfig`, which describes how training is being run,
- a `ModelSpec`, which describes the model's size and inspected layer structure.

The configuration tells us.

- whether we are doing full fine-tuning or LoRA,
- whether training is single-GPU, DDP, ZeRO-2, or ZeRO-3,
- which optimizer family is used,
- which attention backend is active,
- whether checkpointing is enabled,
- the world-size, tensor-parallel, and sequence-parallel choices,
- micro-batch size, sequence length, and dtypes,
- explicit runtime support assumptions and distributed bucket sizes.

The model spec tells us.

- the total parameterization,
- hidden, intermediate, and vocabulary sizes,
- attention widths and head counts,
- which linears are trainable,
- and, when available, explicit parameter metadata for more exact tensor accounting.

6 A Small Notation Guide

Only a few symbols matter for the rest of the report:

- L: number of transformer layers.
- H: hidden size.
- I: intermediate size.
- V: vocabulary size.
- B: micro-batch size per rank.
- S: sequence length.
- $T = B \cdot S$: tokens processed by one rank before any sequence parallel split.
- T_{TP}: tensor-parallel degree.
- D_{P}: data-parallel degree.

When the report refers to a byte width such as b_w or b_g, it simply means "bytes per element for that dtype." For example, bf16 weights imply $b_w = 2$.

7 How To Read The Rest

The remainder of the report follows the same order as the estimator itself:

1. build the resident memory floor,
2. add the retained activations that survive the forward pass,
3. add the phase-local workspace,
4. assemble the forward, backward, and optimizer-step candidates,
5. pick the largest one.

Whenever a term is derived directly from concrete tensors, the report says so. Whenever a term is really a runtime or backend assumption, the report says that too. That distinction is important. It keeps the model honest.

8 Resident State

Resident state is the memory floor that exists before we talk about phase-specific activations or temporary workspace. A useful beginner rule is:

> If the rank still owns the object even while no kernel is actively running, that object belongs in the resident floor.

8.1 The Basic Byte Rule

Every resident tensor follows the same local accounting rule:

$$\text{local bytes} = \text{local elements} \times \text{bytes per element}.$$

The only real question is what "local" means. Tensor parallelism reduces the visible width of some tensors. ZeRO may further shard trainable tensors across the data-parallel group. Once those local element counts are known, the byte computation itself is straightforward.

The two local-count rules that show up throughout the estimator are:

$$n_{\text{local,tp}} \approx \frac{n}{d_{\text{tp}}},$$
$$n_{\text{local,zero}} \approx \frac{n_{\text{local,tp}}}{D_{\text{P}}}.$$

In words: first shard by tensor parallel if that tensor is TP-visible, then shard again by the data-parallel group if the ZeRO stage owns that object in shards. The implementation still rounds to whole elements, but the rounded form does not add much intuition here, so this report shows the simpler proportional version.

8.2 Parameters

Parameter bytes are the cleanest part of the estimator.

Full fine-tuning. All base-model parameters are trainable and therefore matter for:

- resident parameter memory,
- gradient memory,
- optimizer state,
- and, if enabled, master weights.

LoRA. The frozen base model is still resident because the forward pass needs it, but only the synthetic adapter parameters are trainable. That distinction matters: LoRA still pays for the base weights in the resident floor, but it does not pay for full-model gradients or full-model optimizer state.

ZeRO stages. The most important resident distinction is:

- `zero2`: parameters remain replicated, even though gradients and optimizer state are sharded.
- `zero3`: trainable parameters are also sharded across the data-parallel group.

At a high level, the resident parameter term is:

$$M_{\text{params}} = M_{\text{base weights}} + M_{\text{adapter weights}}.$$

For full fine-tuning the adapter term is zero. For LoRA the base weights still exist, but the adapter term is the one that later drives trainable-state objects such as gradients and optimizer state.

8.3 Gradients

Gradient bytes are built from the trainable set, not from the entire parameter set. This is why LoRA can dramatically reduce gradient memory even though the base model remains resident.

For distributed training, the estimator treats gradients as:

- replicated for single-GPU and DDP,
- sharded for ZeRO-2 and ZeRO-3.

That choice is structural. It reflects where gradients live at steady state, not where they may briefly appear during a communication window.

Written compactly:

$$M_{\text{grads}} = \text{local trainable elements} \times \text{gradient bytes per element}.$$

8.4 Optimizer State

Optimizer state is also built from the trainable set. Different optimizers own different amounts of state:

- Adam and AdamW keep two auxiliary tensors per trainable tensor.
- SGD with no momentum keeps essentially none.
- RMSProp and Adagrad keep one state tensor.
- Adafactor is special because its state can be factored instead of dense.

This is one of the places where the estimator is more principled than a simple "optimizer multiplier": it does not assume all optimizers behave the same. The optimizer family changes the actual state objects.

The compact optimizer-state equation is:

$$M_{\text{opt}} = \sum_{p \in \mathcal{P}_{\text{train}}} \rho_{\text{opt}}(p)\, n_{\text{local}}(p)\, b_o^{\star},$$

where $\rho_{\text{opt}}(p)$ is the optimizer-state ratio for parameter p.

8.5 Master Weights

Master weights are optional fp32 copies used by some training setups. When they exist, they are part of the resident floor because they survive across the entire training step.

8.6 Runtime Support

Not every byte on a GPU belongs to a model tensor. CUDA, the caching allocator, NCCL, and DeepSpeed all keep their own support state. The estimator therefore models runtime support separately from tensor memory.

The current model exposes four named support components:

- CUDA context support,
- allocator-pool support,
- NCCL support,
- DeepSpeed support.

If the caller wants full control, the estimator also allows one explicit runtime-support override. The important design choice is not the exact default numbers. It is that these bytes are now represented as named runtime objects instead of being hidden inside one opaque per-mode floor.

That term is:

$$M_{\text{runtime}} = \begin{cases} M_{\text{override}} & \text{if an explicit override is given,} \\ M_{\text{cuda}} + M_{\text{alloc}} + M_{\text{nccl}} + M_{\text{ds}} & \text{otherwise.} \end{cases}$$

8.7 Persistent Backend Buffers

One more resident term appears in some distributed full fine-tuning regimes: persistent backend buffers. These are not model tensors in the usual sense. They stand for backend-managed storage that remains live across phases. The estimator keeps this term explicit because it is real memory, but it is not the same kind of object as a parameter or an activation.

8.8 The Two Resident Floors

After collecting those pieces, the estimator builds two floors:

- the **base floor**, which includes everything resident except gradients,
- the **gradient floor**, which is the base floor plus gradients.

Written compactly:

$$\begin{aligned} R_{\text{base}} &= M_{\text{params}} + M_{\text{opt}} + M_{\text{master}} + M_{\text{runtime}} + M_{\text{persist}}, \\ R_{\text{grad}} &= R_{\text{base}} + M_{\text{grads}}. \end{aligned}$$

These two floors are enough for the phase model:

- forward starts from R_{base},
- backward and optimizer step start from R_{grad}.

9 Retained Activations

This is the part of the model that is easiest to misread, because many people informally use the word “activations” to mean several different things at once.

9.1 Visible Is Not The Same As Retained

The estimator makes a deliberate distinction:

- **hook-visible outputs** are tensors you can see in instrumentation,
- **retained forward state** is the subset that actually survives the end of forward,
- **backward activation state** is what remains alive during backward, including backward-only context.

This distinction is not cosmetic. A hook can see a tensor that is later freed before backward begins. Conversely, a tensor may be small in a hook trace but have a large effect because it is replicated across layers or preserved through checkpoint boundaries.

9.2 The Retained-Forward Proxy

The estimator gathers retained-forward state into one proxy term, M_{proxy}. It is built from named tensor classes rather than one black-box multiplier. The important classes are:

- visible propagation state,
- checkpoint-boundary inputs,
- deduplicated saved linear inputs,
- MLP intermediates when they must survive,
- attention-saved state,
- LoRA low-rank intermediates,
- logits or loss context when needed,
- widened attention-path contexts for architectures whose attention widths are not all equal to the hidden size.

The key design choice here is explicit attribution. The proxy is not supposed to answer "how much forward stuff exists somewhere." It is supposed to answer "which named forward objects are expected to survive into backward."

The compact summary is:

$$M_{\text{proxy}} = M_{\text{visible}} + M_{\text{saved_inputs}} + M_{\text{mlp}} + M_{\text{attn}} + M_{\text{lora}} + M_{\text{checkpoint}} + M_{\text{loss}} + M_{\text{widened}}.$$

Not every regime activates every term, but this is the right way to think about the proxy: it is a sum of named retained object classes.

Full fine-tuning. When the whole stack is trainable and checkpointing is off, retained activations grow roughly like:

$$M_{\text{proxy}}^{\text{full_ft, no ckpt}} \propto L \cdot T_{\text{local}} \cdot (H + I + d_{\text{attn,path}}) \,.$$

That is the expensive regime most people expect: activation pressure grows linearly with depth and with local token count, and the MLP as well as the attention path can both matter.

LoRA. LoRA usually keeps much less activation state because only the targeted adapter paths are trainable:

$$M_{\text{proxy}}^{\text{lora, no ckpt}} \propto L \cdot T_{\text{local}} \cdot (d_{\text{target path}} + r)\,,$$

where r stands for the low-rank adapter width. The important point is not the exact symbol choice. It is that LoRA growth is still linear in depth and token count, but it is tied to the trainable adapter paths rather than the full hidden-plus-MLP stack.

Backend contrast. If the only thing you want to remember is how eager and non-eager attention differ, the comparison can be compressed into one small table:

Backend family	Extra retained-growth term	Interpretation
eager (`standard`)	$L \cdot T_{\text{local}} \cdot S_{\text{eff}} h_{\text{local}}$	The estimator carries an attention-score-like state that grows with effective sequence length and local head count. This is why eager attention becomes much more sequence-sensitive.
non-eager (`sdpa`/`flash2`)	$L_{\text{act}} \cdot T_{\text{local}} \cdot d_{\text{attn,path}}$	The large score-retention term disappears. What remains is width-shaped context from the attention path, so growth stays linear in tokens.

Architecture still matters inside the non-eager family. The widened attention-path terms make some models grow faster than a plain hidden-size story would suggest:

$$M_{\text{widened}} \propto L_{\text{act}} \cdot T_{\text{local}} \cdot d_{\text{widened path}}.$$

So even when two models share the same hidden size, the one with wider attention-path metadata can retain more activation state.

9.3 What Checkpointing Changes

Checkpointing does not magically remove all activation memory. What it changes is the number of layers that are allowed to keep full internal state at once.

The current implementation uses a simple segmentation rule:

- without checkpointing, retained-forward classes can scale with the full layer stack,
- with checkpointing, only one checkpoint segment is allowed to retain its full internal state at a time.

In the current code that effective segment length is one block. That keeps the logic simple and aligns well with the measured checkpointed regimes in the canonical corpus.

Viewed proportionally, checkpointing changes the layer factor much more than the token factor:

$$M_{\text{proxy}}^{\text{ckpt}} \propto L_{\text{act}} \cdot T_{\text{local}} \cdot d_{\text{retained}}, \qquad L_{\text{act}} \ll L.$$

So checkpointing does not make activation memory stop scaling with sequence length. It mainly collapses the number of layers that can keep full retained state at once.

Written compactly:

$$L_{\text{act}} = \begin{cases} L & \text{if checkpointing is off,} \\ 1 & \text{if checkpointing is on.} \end{cases}$$

9.4 Backward Activation State

Backward activation memory is not rebuilt from scratch. The estimator starts from the retained-forward proxy and then adds backward-only context:

$$A_{\text{bwd}} = M_{\text{proxy}} + M_{\text{bwd,extra}}.$$

The extra backward term covers objects that do not need to survive the whole forward pass but do need to exist while gradients are being computed. Examples include residual or normalization context and some saved-input state that only matters once backward begins.

Checkpointed distributed LoRA now uses a deliberately slim backward path. In DDP, ZeRO-2, and ZeRO-3, the forward proxy is the checkpoint boundary plus the loss-state term and the LoRA low-rank intermediates. Backward then adds only the saved linear-input context and the residual or normalization context that must become live once gradients are formed.

Viewed proportionally, backward keeps the same forward-retained base and adds a smaller extra context term:

$$A_{\text{bwd}} \propto M_{\text{proxy}} + T_{\text{local}} \cdot d_{\text{bwd,extra}}.$$

This is why backward often remains larger than forward even after the retained proxy has already been counted.

Earlier versions also added an extra ZeRO-3 checkpointed visible-propagation slice on top of this path. Long-context OLMo 3 replay showed that this term systematically overcounted backward activation state, so the current estimator removes that addend and keeps the checkpointed distributed LoRA path aligned to named retained objects only.

So the two activation views are:

$$\begin{aligned} A_{\text{fwd}} &= M_{\text{proxy}}, \\ A_{\text{bwd}} &= M_{\text{proxy}} + M_{\text{bwd,extra}}. \end{aligned}$$

9.5 Why The Estimator Reports Two Activation Views

The public output exposes both:

- the retained-forward proxy, which is the best analytical match to the measured retained-activation observable,
- and the phase-local activation bucket that is used when a specific phase wins the peak.

This separation keeps the calibration honest. A measured forward-retained proxy should be compared to the estimator's retained-forward proxy, not to the backward activation bucket and not to the public breakdown after a different phase wins.

10 Workspace and Communication Windows

Resident state explains what is already sitting on the rank. Workspace explains what spikes only while a phase is running.

> The estimator treats workspace as a *live window*, not as a sum over every temporary allocation that might appear somewhere in the phase.

That choice is deliberate. Most kernels do not keep every temporary alive at the same time, so summing all of them would be too pessimistic.

10.1 Forward and Backward Workspace

There are four main sources of phase-local workspace:

- attention workspace,
- MLP or backward-kernel workspace,
- communication workspace from tensor or sequence parallelism,
- loss materialization workspace.

Attention backend choice. The estimator uses two different attention workspace models:

- **standard attention** scales with local tokens, local heads, and effective key length because that path really does materialize a larger score-like object,
- **sdpa** and **flash2** use a width-based proxy tied to the largest local attention-path width, because those kernels avoid the eager score-matrix behavior the standard path exhibits.

This is an important modeling choice: the estimator is not saying all attention backends behave the same and then patching them afterward. It starts from the fact that the kernel strategies are different.

The two compact formulas are:

$$W_{\text{attn,fwd}}^{\text{standard}} \propto T_{\text{local}} \cdot h_{\text{local}} \cdot S_{\text{eff}},$$
$$W_{\text{attn,fwd}}^{\text{non-eager}} \propto T_{\text{local}} \cdot d_{\text{attn,path}}.$$

The exact constants differ, but the structural point is the important one: standard attention scales with a score-like object, while `sdpa` and `flash2` scale with the widest local attention path.

Checkpointing. Checkpointing shifts pressure from retained activations into backward-time recompute workspace. In practice that means:

- forward usually gets cheaper,
- backward gets an extra recompute term,
- total peak may move phases rather than simply shrinking everywhere.

Parallel communication. If tensor parallel or sequence parallel is enabled, the estimator adds explicit communication windows for those exchanges. These are local windows. They are not global cluster totals.

10.2 Optimizer Update Scratch

The optimizer also needs its own temporary workspace. The simplest way to think about it is:

> How much temporary storage does one rank need while it is actively applying one update to the trainable set it owns?

For single-GPU and DDP, that scratch scales with the full local trainable set. For sharded ZeRO paths, it may scale with the local shard instead.

The compact form is:

$$W_{\text{update}} \approx \text{local update elements} \times \text{update bytes per element}.$$

10.3 Configured Bucket Objects

Distributed training introduces another kind of workspace: reducer buckets, all-gather buckets, and prefetch buckets. The estimator now models those as explicit configured objects rather than hidden byte caps.

DDP. DDP has one reducer bucket. Its capacity is:

$$B_{\text{ddp}}^{\text{cap}} = E_{\text{ddp}} \cdot b_g^{\star},$$

where E_{ddp} is the configured bucket size in gradient elements.

ZeRO. ZeRO has three relevant bucket objects:

- an all-gather bucket,
- a reduce bucket,
- a prefetch bucket.

Their capacities come from configured element counts as well. Because the estimator is per-rank, it converts those distributed bucket sizes into local rank capacities by dividing over the data-parallel group before multiplying by the appropriate dtype width.

The capacity equations are:

$$\begin{aligned} B_{\text{zero,ag}}^{\text{cap}} &= \frac{E_{\text{zero}}}{D_{\text{P}}} \cdot b_p^{\star}, \\ B_{\text{zero,red}}^{\text{cap}} &= \frac{E_{\text{zero}}}{D_{\text{P}}} \cdot b_g^{\star}, \\ B_{\text{zero,pf}}^{\text{cap}} &= \frac{E_{\text{prefetch}}}{D_{\text{P}}} \cdot b_p^{\star}. \end{aligned}$$

Again, the implementation rounds to whole elements, but the simplified form is easier to read and captures the real regime difference: these capacities shrink with the data-parallel degree.

Capacity versus live bytes. One subtle point matters here: the configured bucket size describes the capacity of the object, not the amount of useful payload that always fills it. The estimator therefore clips each bucket to the local payload that can actually occupy it. A rank cannot actively hold more gradient bytes than it owns, even if the configured reducer bucket is larger than that.

That idea is captured by:

$$B_{\text{active}} = \min(B_{\text{capacity}}, M_{\text{local payload}}).$$

10.4 DDP and ZeRO Windows In Words

Once those bucket objects exist, the distributed windows become easier to read.

DDP. DDP contributes:

- the reducer bucket itself,
- and a communication overlap window that looks like "local gradients plus the reducer bucket."

ZeRO. ZeRO contributes three windows:

- a **fetch window**, which is the local trainable payload plus the all-gather and prefetch buckets,
- an **update window**, which is the optimizer update scratch,
- a **communication window**, which is the local gradient payload plus the reduce bucket.

The estimator then takes the largest of those windows instead of summing them. That is the right abstraction for "what is the largest temporary distributed object live on this rank during the optimizer phase?"

The same idea in compact notation is:

$$\begin{aligned} W_{\text{zero,fetch}} &= M_{\text{train}} + B_{\text{zero,ag}} + B_{\text{zero,pf}}, \\ W_{\text{zero,comm}} &= M_{\text{grads}} + B_{\text{zero,red}}, \\ W_{\text{opt,zero}} &= \max(W_{\text{zero,fetch}}, W_{\text{update}}, B_{\text{zero,red}}) \,. \end{aligned}$$

10.5 Why One ZeRO-2 Full-FT Lifetime Path Remains

One stage-specific path still remains for non-checkpointed `zero2 full_ft`. It is not there because the code wants a special case for its own sake. It is there because the canonical replay still shows a repeatable state lifetime pattern in that regime.

The practical interpretation is:

- in backward, some gradient-related state behaves as if two copies of the local gradient payload can overlap with the reduce bucket,
- in optimizer step, the live window can include the master shard, local gradients, update scratch, parameter partition support, and the ZeRO bucket objects at the same time.

The report keeps this path explicit rather than pretending it comes from the generic ZeRO window model. That is the more honest presentation.

The retained explicit lifetime path is:

$$W_{\text{bwd}} \leftarrow \max(W_{\text{bwd}}, 2M_{\text{grads}} + B_{\text{zero,red}}) \,,$$

$$\begin{aligned} W_{\text{opt}} \leftarrow \max\Big(W_{\text{opt}}, M_{\text{master,shard}} + 2M_{\text{grads}} + W_{\text{update}} \\ + M_{\text{param_partition}} + B_{\text{zero,ag}} + B_{\text{zero,pf}} + B_{\text{zero,red}}\Big). \end{aligned}$$

10.6 Backward-End State

The optimizer step does not start from a perfectly clean floor. Some state is still live at the boundary between backward and the step. The estimator calls that the *backward-end state*.

The intuition is simple:

- single-GPU has no distributed bucket object to keep alive here,
- DDP typically carries local gradients plus the reducer bucket,

- ZeRO-2 carries local gradients plus the reduce bucket,
- ZeRO-3 carries local gradients plus the all-gather-side support object needed for the next step window.

This term matters because a lot of optimizer-step underestimation comes from forgetting that the step does not begin from an empty temporary state.

The compact regime summary is:

$$E_{\text{bwd}} = \begin{cases} 0 & \text{single_gpu}, \\ M_{\text{grads}} + B_{\text{ddp}} & \text{ddp}, \\ M_{\text{grads}} + B_{\text{zero,red}} & \text{zero2}, \\ M_{\text{grads}} + B_{\text{zero,ag}} & \text{zero3}. \end{cases}$$

11 Phase Assembly

At this point the estimator has everything it needs:

- resident floors,
- retained activations,
- backward-end state,
- phase-local workspace,
- optimizer-step carryover.

The three candidate peaks are then:

$$\begin{aligned} M_{\text{forward}} &= R_{\text{base}} + A_{\text{fwd}} + W_{\text{fwd}}, \\ M_{\text{backward}} &= R_{\text{grad}} + A_{\text{bwd}} + W_{\text{bwd}}, \\ M_{\text{optimizer}} &= R_{\text{grad}} + E_{\text{bwd}} + W_{\text{opt}} + C_{\text{opt}}. \end{aligned}$$

11.1 Optimizer Carryover

Carryover is the least elegant part of the estimator, so it deserves a plain language explanation.

> Carryover means memory that is not part of the clean resident floor but still has not been released when the optimizer step begins.

Different regimes carry different things:

- some non-checkpointed ZeRO-2 full-FT paths carry a meaningful fraction of the backward workspace or retained-forward state into the step,
- checkpointed full-FT ZeRO paths mainly carry backward activation state,
- checkpointed single-GPU LoRA can carry both retained-forward state and some backward workspace into the step,

- checkpointed distributed LoRA can also carry allocator reserve built up by loss materialization and backward recompute, even when the clean resident and activation terms look small at optimizer-step entry,
- many simpler regimes carry essentially nothing extra.

The estimator keeps this logic as a short regime table in code because the lifetimes really are regime-dependent. The important thing is that the report states this openly instead of burying it in a single unexplained formula.

The newest example of that is checkpointed distributed LoRA. The current heuristic adds the retained-forward proxy, one loss-workspace copy, the backward workspace, and the small checkpoint-boundary or saved-input contexts that tend to keep allocator reserve warm on long-context DDP and ZeRO runs. That term is empirical rather than first-principles allocator simulation, but it materially improves replay on the long-context OLMo 3 corpus without changing the cleaner resident-state accounting elsewhere.

A compact generic view is:

$$C_{\text{opt}} = M_{\text{carry,extra}},$$

where $M_{\text{carry,extra}}$ means "state still live at optimizer-step entry that has not already been counted in $R_{\text{grad}} + E_{\text{bwd}}$."

11.2 Public Breakdown Semantics

The public memory breakdown is aligned to the winning phase, not to every phase at once.

That means:

- parameter, gradient, optimizer-state, and runtime-reserve terms always show up directly,
- activation and transient terms depend on which phase actually wins,
- if optimizer step wins, the public activation bucket is intentionally set to zero and the phase-local excess is shown as transient memory instead.

This choice keeps the public output readable while still allowing phase-aligned calibration to compare like with like internally.

12 Empirical Assumptions That Remain

Most of the estimator is explicit tensor accounting. A few pieces still remain as measured or runtime-driven assumptions:

- the attention workspace formulas are still proxies for kernel behavior, not exact allocator traces,
- runtime support is a named support model, not a full CUDA allocator simulation,
- checkpointed distributed LoRA optimizer carryover is still a calibrated allocator-lifetime heuristic rather than a full reserve-pool simulator,
- one stage-specific ZeRO-2 full-FT lifetime path remains because it still improves replay quality materially.

Those assumptions are kept narrow on purpose. The estimator works best when the core of the model stays about named objects and lifetimes, and only the truly backend-shaped pieces remain empirical.

13 Quantitative Accuracy

The current canonical replay covers 179 rows and skips 151 rows that still fail for artifact or support reasons.

$$\begin{aligned}
\text{Global MAE} &= 1.099 \text{ GiB} \\
\text{Mean Absolute Relative Error} &= 10.5\% \\
\text{Median Absolute Relative Error} &= 6.2\% \\
\text{Phase Match Rate} &= 64.8\% \\
\text{Retained-Forward Proxy MAE} &= 0.577 \text{ GiB}
\end{aligned}$$

13.1 Phase Confusion

Measured	Estimated	Count
backward	backward	58
backward	optimizer_step	3
optimizer_step	backward	60
optimizer_step	optimizer_step	58

13.2 Targeted Slices

Slice	Count	Global MAE	Phase Match
Non-Eager Long-Seq Full FT	1	13.451 GiB	100.0%
Non-Eager Long-Seq LoRA	12	1.691 GiB	66.7%
ZeRO-2	68	1.365 GiB	97.1%
ZeRO-3	0	0.000 GiB	0.0%